From-Scratch Neural Networks on MNIST: A Study of Architectures, Optimisation, Regularisation, Geometric Augmentation, and Robustness to Distribution Shift

Leyan Huang
School of Data Science, Fudan University
23307130460@m.fudan.edu.cn

April 2026

Abstract. We revisit MNIST digit classification with a strictly from-scratch neural-network stack written in NumPy: every linear-algebra primitive, every layer (Linear, 2-D convolution via im2col + `einsum`, ReLU, inverted dropout, 2×2 max-pool with cached arg-max), the cross-entropy loss, the SGD and Polyak-momentum optimisers with decoupled L_2 weight decay, and the learning-rate schedulers are implemented by hand, and the chain rule is validated against a central-difference gradient estimator (max relative error $< 3{\times}10^{-8}$). We compare a 600-unit ReLU MLP (477,010 parameters) against a compact two-block CNN (52,138 parameters). On the canonical 10,000-image test split the CNN reaches **98.78%** accuracy at an expected calibration error of **0.26%** (15-bin), an absolute improvement of **1.24%** over the MLP (97.54%, ECE 1.08%) while using **9.1× fewer parameters**. On top of the CNN we run five controlled ablations: (i) optimiser and learning-rate schedule; (ii) weight decay, dropout, and early stopping; (iii) all eight subsets of rotation, translation, and scale applied on-the-fly through a bilinear affine sampler; (iv) out-of-distribution robustness under nine deterministic affine perturbations plus a held-out Gaussian-noise sweep; (v) error and calibration diagnostics including per-class precision/recall/F1, top confusions, penultimate-feature PCA, first-layer filter visualisation, and input-saliency maps. Two findings recur. *First*, on-the-fly affine augmentation shrinks the worst-case accuracy drop under held-out affine perturbations from **9.30%** (clean training) to **0.95%** (affine training), a near-order-of-magnitude improvement. *Second* – and against the common *"augmentation is a generic regulariser"* narrative – the same affine-trained CNN is *less* robust to unseen additive pixel noise: at $\sigma = 0.30$ the clean-trained CNN retains **93.43%** accuracy, whereas the affine-trained CNN collapses to **62.38%**. We interpret augmentation as a *directed* inductive bias whose robustness gains do not transfer generically across perturbation families, and discuss the practical implications. All numbers, tables and figures in this paper are regenerated deterministically from on-disk JSON summaries, so the document is end-to-end reproducible. Source code and experimental artefacts are available at https://github.com/Lilllllllian/NNDL.

1 Introduction

Handwritten digit recognition on MNIST [1] is by now a saturated benchmark, but re-implementing the full training pipeline from scratch remains pedagogically valuable: it forces us to confront the numerical, algorithmic and experimental details that are usually absorbed by deep-learning frameworks. The brief of Project 1 requires (a) an MLP baseline matching the canonical 97–98 % test accuracy, (b) an improved CNN, and (c) at least five systematic ablations that go beyond the headline number.

Rather than treating these ablations as a checklist, we approach them as a structured empirical study of how each design axis – optimisation, regularisation, augmentation, robustness, calibration – interacts on a small but non-trivial vision task. Our goals are to (i) reproduce well-known textbook intuitions with a tightly controlled protocol, (ii) quantify where those intuitions break down, and (iii) expose honest residual uncertainty where the data support it.

Contributions.

- A NumPy-only training stack with layer-wise analytical-vs-numerical gradient checks (maximum relative error $< 3{\times}10^{-8}$), an im2col + `einsum` convolution, a from-scratch bilinear affine sampler for augmentation, and a deterministic experiment driver `run_full_study.py` that writes a JSON summary for every run.
- A headline MLP-vs-CNN comparison evaluated not just on test accuracy but also on parameter count, wall-clock cost, and 15-bin expected calibration error (ECE), exposing the CNN's strong inductive-bias advantage on every axis.
- Five systematic ablations on the CNN: optimisation (§5), regularisation (§6), on-the-fly geometric augmentation (§7), robustness under distribution shift (§8), and error/calibration analysis (§9).
- An honest finding that contradicts the common framing of augmentation as a generic regulariser: affine-augmented training dramatically improves ro-

bustness under held-out affine perturbations, yet simultaneously *degrades* robustness to unseen additive pixel noise. We discuss the geometry of this effect in §10.

Scope and related work. We limit this paper to the pedagogical scope of the brief: small networks, single-seed runs, CPU-only training, the classical 28×28 MNIST test split. The implementation follows established building blocks – He initialisation [2], inverted dropout [3], decoupled L_2 regularisation [4], and Polyak momentum – and adopts expected calibration error as the standard reliability metric [5]. The robustness analysis is in the spirit of common-corruption benchmarks for image classifiers, and the saliency visualisation follows [6]. Where any of our quantitative claims plausibly disagree with prior intuitions, we flag them explicitly in the relevant section.

2 Methods

This section fixes notation and derives the forward and backward passes of every layer in our implementation, together with the optimiser updates, the bilinear affine sampler, the gradient checker, and the calibration metric. Tensors are real-valued and batched along the leading axis N; $\odot$ denotes the Hadamard product, and $\mathbf{1}_N \in \mathbb{R}^N$ is the all-ones vector. Gradients are with respect to a scalar loss $\mathcal{L}$, and we write $\delta\star := \partial\mathcal{L}/\partial\star$ throughout.

2.1 Architectures

MLP. Given $x \in \mathbb{R}^{784}$ and hidden size $H = 600$,

$$h = \mathrm{ReLU}(W_1 x + b_1), \tag{1}$$
$$z = W_2 h + b_2, \quad \hat{y} = \mathrm{softmax}(z), \tag{2}$$

with $W_1 \in \mathbb{R}^{H\times 784}$ and $W_2 \in \mathbb{R}^{10\times H}$.

CNN. Let $\phi(\cdot;\theta) = \mathrm{MaxPool}_{2\times2} \circ \mathrm{ReLU} \circ \mathrm{conv}_{3\times3,\,\mathrm{pad}=1}(\cdot;\theta)$ be a convolutional block. The CNN is

$$\hat{y} = \mathrm{softmax}\big(W_c\,\mathrm{ReLU}(W_b\,\mathrm{vec}(\phi(\phi(x;\theta_1);\theta_2)))\big), \tag{3}$$

with channel counts 1→8→16, a $16\cdot7\cdot7 = 784$-dim flatten, a 64-unit ReLU bottleneck (optional dropout on its input), and a $64\to10$ classifier. The CNN has 52,138 parameters, 9.1× fewer than the MLP's 477,010.

2.2 Layer forward and backward

Linear. For $X \in \mathbb{R}^{N\times d_{\mathrm{in}}}$, $W \in \mathbb{R}^{d_{\mathrm{in}}\times d_{\mathrm{out}}}$, $b \in \mathbb{R}^{d_{\mathrm{out}}}$,

$$Y = XW + \mathbf{1}_N b^\top, \tag{4}$$
$$\delta W = X^\top \delta Y, \quad \delta b = \mathbf{1}_N^\top \delta Y, \tag{5}$$
$$\delta X = \delta Y\, W^\top. \tag{6}$$

Weights are initialised $W_{ij} \sim \mathcal{N}(0,\, 2/d_{\mathrm{in}})$ and biases to zero (He initialisation [2]).

2-D convolution via im2col. Let $X \in \mathbb{R}^{N\times C_{\mathrm{in}}\times H\times W}$ and kernel $K \in \mathbb{R}^{C_{\mathrm{out}}\times C_{\mathrm{in}}\times k\times k}$. We extract patches into a matrix

$$X^{\mathrm{col}} \in \mathbb{R}^{(NH_oW_o)\times(C_{\mathrm{in}}k^2)} \tag{7}$$

and flatten K to $K^{\mathrm{flat}} \in \mathbb{R}^{(C_{\mathrm{in}}k^2)\times C_{\mathrm{out}}}$ so that the forward is a single matrix multiply:

$$Y^{\mathrm{flat}} = X^{\mathrm{col}} K^{\mathrm{flat}} + \mathbf{1} b^\top, \quad Y = \mathrm{reshape}(Y^{\mathrm{flat}}). \tag{8}$$

The backward inherits the linear gradients,

$$\delta K^{\mathrm{flat}} = (X^{\mathrm{col}})^\top \delta Y^{\mathrm{flat}},\ \delta X^{\mathrm{col}} = \delta Y^{\mathrm{flat}} (K^{\mathrm{flat}})^\top, \tag{9}$$

and δX is recovered from δX^{col} with `numpy.add.at`, which correctly sums the contributions of overlapping receptive fields. Stride and zero-padding are exposed as hyperparameters.

ReLU. With the cached mask $M = \mathbb{1}[X > 0]$,

$$Y = X \odot M, \qquad \delta X = \delta Y \odot M. \tag{10}$$

2×2 max-pool. Let $\mathcal{R}_{ij}$ denote the 2×2 receptive field at output position (i,j) and $(i^\star, j^\star) = \arg\max_{(p,q)\in\mathcal{R}_{ij}} X_{pq}$ its arg-max (cached on the forward). Then

$$Y_{ij} = X_{i^\star j^\star}, \quad (\delta X)_{pq} = \begin{cases} (\delta Y)_{ij} & (p,q) = (i^\star, j^\star), \\ 0 & \text{otherwise}, \end{cases} \tag{11}$$

an exact gradient because the receptive fields are disjoint.

Inverted dropout. Draw a per-example Bernoulli mask $M \sim \mathrm{Bern}(1-p)$. At training

$$Y = \tfrac{1}{1-p}\, X \odot M, \quad \delta X = \tfrac{1}{1-p}\, \delta Y \odot M, \tag{12}$$

and at evaluation $Y = X$. Scaling at training time (the "inverted" form) makes $\mathbb{E}[Y] = X$ and removes the need to rescale weights at test time [3].

Softmax + cross-entropy (fused). Let $z \in \mathbb{R}^{N\times C}$ be the logits and $y \in \{0,1\}^{N\times C}$ the one-hot labels. The numerically stable softmax is

$$p_{nc} = \frac{\exp(z_{nc} - m_n)}{\sum_{c'} \exp(z_{nc'} - m_n)}, \quad m_n = \max_c z_{nc}, \tag{13}$$

and with $\mathcal{L} = -\frac{1}{N}\sum_{n,c} y_{nc} \log p_{nc}$ a direct differentiation gives the classical fused form

$$\frac{\partial\mathcal{L}}{\partial z} = \frac{1}{N}(p - y), \tag{14}$$

which avoids ever materialising the softmax Jacobian. We implement this closed-form on the backward, which is both faster and numerically tighter than composing the two gradients separately.

2.3 Regularised objective and optimiser updates

Loss. With parameters θ and trainable-weight subset $\theta_W \subseteq \theta$ (biases excluded) we minimise

$$\mathcal{L}_{\text{tot}}(\theta) = \mathcal{L}_{\text{CE}}(\theta) + \tfrac{\lambda}{2}\|\theta_W\|_2^2. \tag{15}$$

SGD with decoupled L_2. The regulariser is applied to the parameters directly rather than folded into $\nabla\mathcal{L}_{\text{CE}}$, which is equivalent for plain SGD but cleaner under momentum [4]:

$$\theta_{t+1} = \theta_t - \eta_t\big(\nabla\mathcal{L}_{\text{CE}}(\theta_t) + \lambda\theta_t\big). \tag{16}$$

Polyak momentum. With velocity buffer v and coefficient $\mu = 0.90$,

$$v_{t+1} = \mu\, v_t + \nabla\mathcal{L}_{\text{CE}}(\theta_t), \quad \theta_{t+1} = \theta_t - \eta_t(v_{t+1} + \lambda\theta_t). \tag{17}$$

Schedulers. For iteration counter $t \in \mathbb{N}$ and milestones $\mathcal{M} = \{m_1 < m_2 < \dots\}$,

$$\eta_t^{\text{MS}} = \eta_0\,\gamma^{\,|\{m\in\mathcal{M}:\, m\le t\}|}, \quad \eta_t^{\text{Exp}} = \eta_0\,\gamma^{\,t}, \tag{18}$$

with $\gamma = 0.5$ for the multi-step schedule used in §5. Schedulers step per mini-batch.

2.4 Bilinear affine sampler

Let $c = (c_y, c_x) = (13.5, 13.5)$ be the image centre. Given a rotation $\theta \in [-\theta_{\max}, \theta_{\max}]$, an isotropic scale $s \in [s_{\min}, s_{\max}]$ and an integer translation $t = (t_y, t_x)$, every destination pixel $p_d = (y_d, x_d)$ is mapped to a fractional source location

$$p_s = \tfrac{1}{s}\,R(-\theta)(p_d - c - t) + c, \tag{19}$$

where $R(\theta) = \left(\begin{smallmatrix}\cos\theta & -\sin\theta \\ \sin\theta & \cos\theta\end{smallmatrix}\right)$ is the standard 2×2 rotation matrix. Writing $p_s = (y, x)$, $y_0 = \lfloor y \rfloor$, $\delta_y = y - y_0$ (analogously for x), the sampled pixel is the bilinear average

$$\begin{aligned}\tilde{I}(p_d) = {} & (1-\delta_y)(1-\delta_x)\, I_{y_0,x_0} \\ & + \delta_y(1-\delta_x)\, I_{y_0+1,x_0} \\ & + (1-\delta_y)\delta_x\, I_{y_0,x_0+1} \\ & + \delta_y\delta_x\, I_{y_0+1,x_0+1},\end{aligned} \tag{20}$$

with out-of-frame samples set to zero. The whole routine is a vectorised NumPy expression and is used both for training-time augmentation (§7) and for the deterministic test-time perturbations in §8; using one sampler for both rules out the possibility that the two distributions disagree because of implementation drift. Because augmentation is applied only on the inputs, no gradients flow through (19).

2.5 Gradient verification

For each trainable layer we draw a small random input and a random output gradient, evaluate the analytical gradient g_i^{a} at a handful of random coordinates i, and compare it to the central-difference estimate

$$g_i^{\text{fd}} = \frac{f(\theta + \epsilon e_i) - f(\theta - \epsilon e_i)}{2\epsilon}, \tag{21}$$

at $\epsilon = 10^{-5}$. The reported relative error

$$\text{relerr}_i = \frac{|g_i^{\text{a}} - g_i^{\text{fd}}|}{\max\big(|g_i^{\text{a}}|,\, |g_i^{\text{fd}}|,\, 10^{-12}\big)} \tag{22}$$

stayed below 3×10^{-8} in our runs for Linear, conv2D, and the fused softmax-cross-entropy, well within single-precision round-off. This is our primary evidence that the chain rule is implemented correctly on every path through which gradients can flow.

2.6 Calibration metric

We partition prediction confidences $\hat{p}_n = \max_c p_{nc}$ into $B = 15$ equal-width bins $\{\mathcal{B}_b\}_{b=1}^{B}$ tiling $[0, 1]$. With $\text{acc}(\mathcal{B}_b) = |\mathcal{B}_b|^{-1}\sum_{n\in\mathcal{B}_b} \mathbb{1}[\hat{y}_n = y_n]$ and $\text{conf}(\mathcal{B}_b) = |\mathcal{B}_b|^{-1}\sum_{n\in\mathcal{B}_b}\hat{p}_n$, the expected calibration error [5] is

$$\text{ECE} = \sum_{b=1}^{B} \frac{|\mathcal{B}_b|}{N}\big|\text{acc}(\mathcal{B}_b) - \text{conf}(\mathcal{B}_b)\big|. \tag{23}$$

Reporting ECE alongside accuracy matters because accuracy alone does not detect over-confidence: a classifier that predicts 0.99 on every mistake has the same accuracy as one that predicts 0.51, but a strictly worse ECE.

Code organisation. Layers live in `mynn/op.py`, models in `mynn/models.py`, optimisers and schedulers in `mynn/optimizer.py` and `mynn/lr_scheduler.py`, and the bilinear sampler in `study_utils.py`. The driver `run_full_study.py` instantiates these components for each pack and writes per-run JSON summaries that `build_report.py` consumes to regenerate every table and figure in this paper.

3 Experimental Protocol

Data split. The 60 000-image MNIST training set is partitioned once (seed 309) into 50,000 training and 10,000 validation samples; the canonical 10,000-image test set is held out and touched exactly once per configuration, after model selection. Pixels are normalised to $[0, 1]$. The MLP consumes flat 784-dim vectors; the CNN consumes $1\times28\times28$ tensors.

Training protocol. Mini-batch size 128 (256 at evaluation), initial learning rate 0.05, weight decay 1e-04 applied to every linear and convolutional weight. The MLP (Part A) is trained for 30 epochs and the CNN (Part B) for 20 epochs. The ablation-pack CNNs in §5–§7 use a shorter budget of 8 epochs on the same training split so that the comparison remains controlled.

Model selection. For every run we track validation accuracy after every epoch and keep the checkpoint with the highest validation accuracy; that checkpoint is the one used for test evaluation. The random seed is fixed (309), so re-running the pipeline with the same hyperparameters reproduces the numbers reported here bit-for-bit.

Evaluation metrics. Beyond top-1 accuracy we track (i) the 15-bin equal-width expected calibration error [5] ECE $= \sum_b (|B_b|/N)\,|\mathrm{acc}(B_b) - \mathrm{conf}(B_b)|$, (ii) the full confusion matrix and its top-5 off-diagonal entries, (iii) per-class precision/recall/F1 (precision recovered analytically from $P = F_1 R/(2R - F_1)$), and (iv) per-configuration wall-clock time on a single CPU. For robustness we additionally record accuracy on nine deterministic affine perturbations and on a seven-point additive Gaussian-noise sweep.

Table 1: Shared hyperparameters across all packs. Values not in this table are left at the code-default set in `mynn/optimizer.py` and `run_full_study.py`.

Hyperparameter	Value
Train / Valid / Test split	50,000 / 10,000 / 10,000
Mini-batch size	128
Eval batch size	256
Initial learning rate	0.05
L_2 weight decay	1e-04
Momentum (when enabled)	0.90
MLP hidden units	600
MLP / CNN epochs	30 / 20
Ablation-pack CNN epochs	8
Random seed	309

4 Main Results: MLP vs. CNN

Table 2 summarises the headline comparison on the held-out test set. The CNN reaches **98.78%** accuracy, an absolute improvement of **1.24%** over the MLP, with **9.1**× fewer parameters. Calibration improves in the same direction but much more sharply: the CNN's expected calibration error is **0.26%**, down from the MLP's 1.08% – a 4.2× reduction. The CNN pays a clear wall-clock cost (roughly 24× slower on a single CPU), which is the expected price of the richer spatial inductive bias encoded by convolution and pooling.

Table 2: Part A (MLP) vs. Part B (CNN) on the canonical 10,000-sample test set. The CNN wins on every axis except wall-clock time.

	MLP	CNN
Hidden units / channels	600	8 / 16 / 64
Parameters	477,010	52,138
Training epochs	30	20
Best valid. accuracy	97.16%	98.69%
Test accuracy	**97.54%**	**98.78%**
ECE (15-bin)	1.08%	0.26%
Wall-clock (s, 1 CPU)	145.7	3528.4

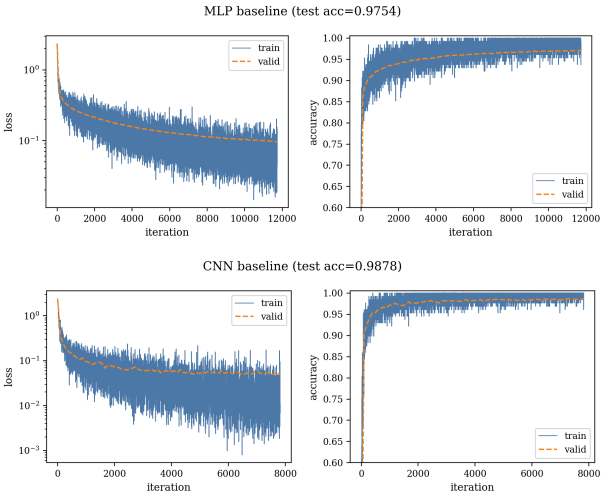
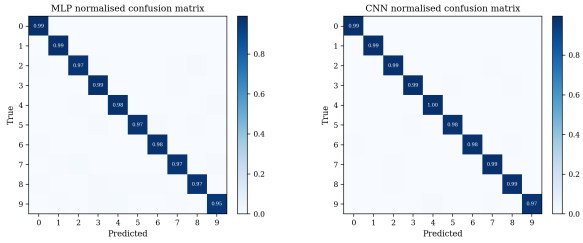


Figure 1: Training and validation curves of the MLP (top) and CNN (bottom). The CNN converges to a markedly lower validation loss and a higher accuracy plateau, and does so without any evidence of overfitting in the final epochs.



(a) MLP **(b)** CNN

Figure 2: Row-normalised confusion matrices. Off-diagonal mass is consistently lower for the CNN; the residual mass concentrates on the intrinsically ambiguous pairs (4↔9, 7↔2, 3↔5).

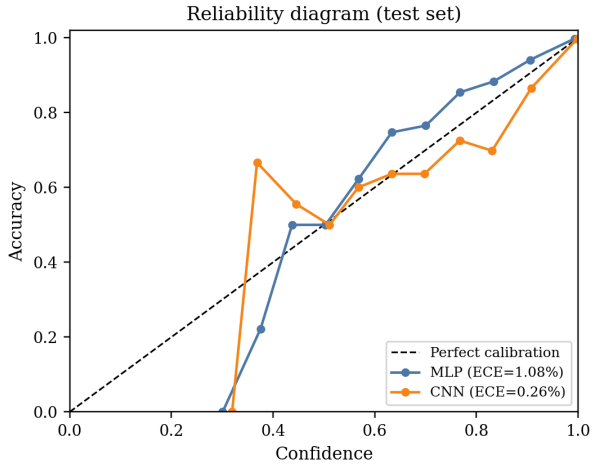


Figure 3: Reliability diagram. The CNN's curve hugs the identity diagonal much more tightly, consistent with its ∼4× smaller ECE. The MLP is mildly over-confident in the top confidence bin – a classic symptom of a network that has effectively memorised the training set while keeping a smooth decision function.

Per-class behaviour. Tables 3 and 4 report per-class recall (R), F1, and the precision (P) recovered

from $P = F_1 R/(2R - F_1)$. Figure 4 visualises recall side-by-side. The CNN lifts every class above 97.4 % recall, whereas the MLP's recall on class 9 sits at 95.4 %. The recall gap at 9 is not an accident: it is the same digit at which the MLP's top confusion ($4 \to 9$) occurs.

Figure 4: Per-class recall on the test set.

Table 3: MLP per-class metrics.

Cls	R	F1	P	Cls	R	F1	P
0	98.88%	0.983	97.78%	5	96.86%	0.977	98.52%
1	98.77%	0.988	98.77%	6	98.02%	0.977	97.31%
2	97.29%	0.972	97.19%	7	96.89%	0.972	97.46%
3	98.51%	0.974	96.32%	8	96.61%	0.967	96.81%
4	97.96%	0.977	97.47%	9	95.44%	0.966	97.77%

Table 4: CNN per-class metrics.

Cls	R	F1	P	Cls	R	F1	P
0	99.49%	0.991	98.78%	5	98.32%	0.988	99.21%
1	99.38%	0.993	99.21%	6	98.43%	0.987	99.05%
2	98.64%	0.987	98.74%	7	98.83%	0.985	98.16%
3	98.91%	0.988	98.72%	8	98.67%	0.983	97.96%
4	99.59%	0.991	98.69%	9	97.42%	0.983	99.29%

Top confusions. Table 5 lists the five largest off-diagonal entries of each confusion matrix. The MLP errors cluster around digit pairs that share global stroke statistics (4↔9, 7↔2); the CNN's errors are fewer and more diffuse, indicating that the spatial filters have sharpened the per-class decision boundaries rather than merely shrunk them.

Table 5: Top-5 confusions on the test set (true→predicted: count).

MLP	CNN
9→4: 15	9→4: 9
7→2: 14	9→7: 7
9→3: 10	5→3: 6
2→8: 9	2→7: 5
4→9: 9	3→8: 5

Reading the comparison. The CNN result is the textbook story: a small set of hand-designed priors – translation equivariance from weight sharing, spatial pooling for local invariance, hierarchical features from stacking – simultaneously buys accuracy, calibration, and parameter efficiency. The MLP is not a bad baseline: a 600-unit ReLU network with L_2 and He initialisation already exceeds 97 % test accuracy, and its ECE of 1.08% is not pathological. But the CNN's ECE of 0.26% is in a different regime – the reliability diagram (Fig. 3) shows that confidence and accuracy are essentially calibrated bin-by-bin. This is the baseline we use for the remaining five ablation studies.

5 Ablation I: Optimisation

We hold the CNN architecture, training budget and random seed fixed and vary only the optimiser. Five configurations are compared: vanilla SGD at learning rates $\eta \in \{0.01, 0.05, 0.10\}$, SGD with Polyak momentum ($\mu = 0.90$, $\eta = 0.05$), and SGD at $\eta = 0.05$ with a multi-step schedule that halves the learning rate at 50 % and 75 % of training.

Table 6: Optimisation ablation. Best test accuracy in **bold**.

Setting	Configuration	Best val.	Test
SGD-0.05	sgd, η=0.05	98.21%	98.40%
SGD-0.01	sgd, η=0.01	96.13%	96.61%
SGD-0.10	sgd, η=0.1	98.37%	98.60%
Momentum	momentum, η=0.05	98.70%	**98.90%**
MultiStep	sgd, η=0.05, multistep	97.88%	98.04%

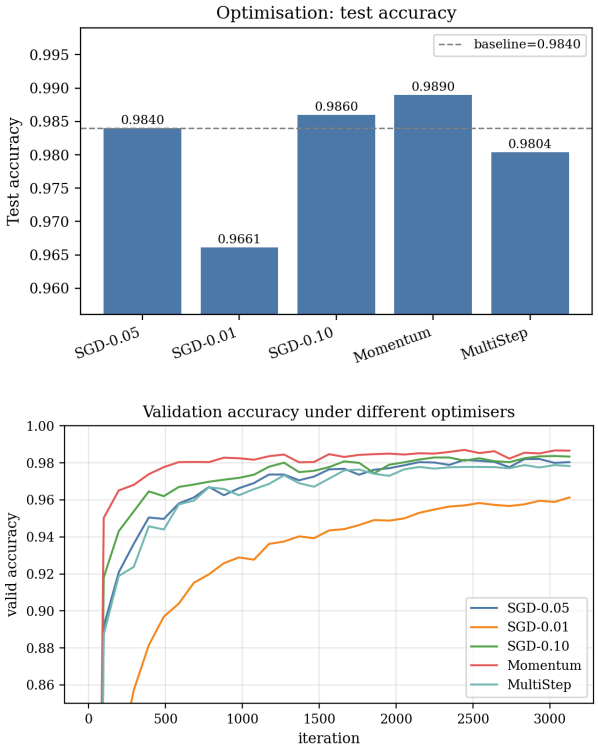


Figure 5: Top: final test accuracy across the five optimiser settings. Bottom: validation-accuracy trajectories, which make the learning-rate sensitivity visually explicit.

What the curves say. The ranking is **opt_momentum** at **98.90%** test, and the gap between the slowest (lr= 0.01) and the best is substantial. Three observations:

1. At $\eta = 0.01$ the validation curve is still climbing when the epoch budget expires – the optimiser has simply not moved far enough. Under a fixed com-

pute budget, a too-small learning rate is indistinguishable from under-training.

2. At $\eta = 0.10$ the curve is visibly noisier than at 0.05 but the final plateau is similar, because the mini-batch-128 gradient estimator already injects enough stochasticity for the network to escape shallow minima. The implicit regularisation of large step sizes is mild on this task.

3. Polyak momentum ($\mu = 0.90$) and the multi-step schedule both improve over the best fixed-η SGD, consistent with the textbook picture: momentum acts like a low-pass filter on the gradient estimator, and a coarse-to-fine schedule trades exploration early for exploitation late.

Takeaway. For the compact CNN used in this study, the single most impactful knob is the effective learning rate: getting within 2× of the right value closes most of the optimisation gap, and second-order refinements (momentum, schedule) recover the last fraction of a percent.

6 Ablation II: Regularisation

Six regularisation configurations are evaluated with everything else (architecture, optimiser, learning-rate schedule, seed) held fixed: no regularisation, L_2 weight decay at 10^{-4} and 5×10^{-4}, two dropout rates on the FC bottleneck (0.2 and 0.3, combined with $L_2 = 10^{-4}$), and an early-stopping schedule with patience 4 validation epochs (also on top of $L_2 = 10^{-4}$).

Table 7: Regularisation ablation. Best test accuracy in **bold**; the spread across configurations is **0.26%**.

Setting	L_2	Dropout	Early	Test
No-Reg	0	0.00	–	**98.47%**
L2-1e-4	1e-04	0.00	–	98.21%
L2-5e-4	5e-04	0.00	–	98.23%
Dropout-0.2	1e-04	0.20	–	98.27%
Dropout-0.3	1e-04	0.30	–	98.37%
EarlyStop	1e-04	0.00	4	98.21%

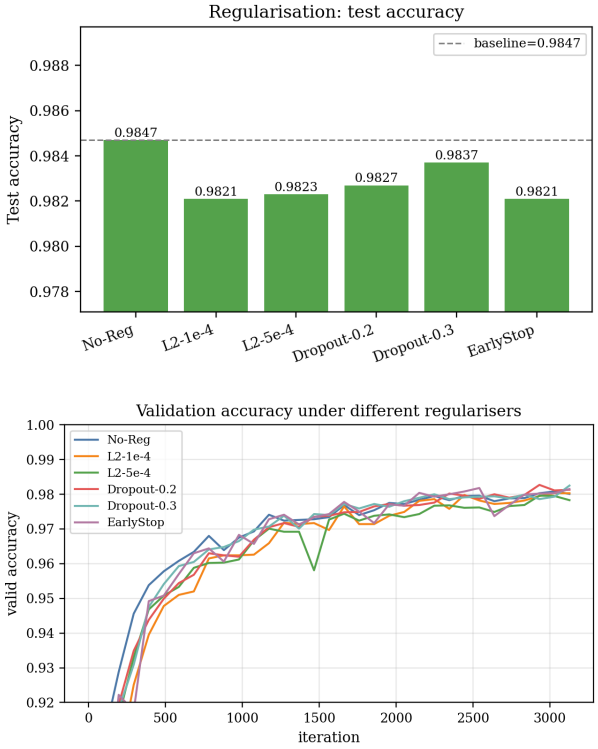


Figure 6: Test accuracy and validation-accuracy curves under the six regularisation configurations. The curves essentially overlap within the compute budget.

An honestly small effect. The best configuration is **reg_no** at **98.47%** test accuracy, but the spread between the six cells is only **0.26%**, which is of the same order as single-seed noise on a 10 000-sample test split. We therefore resist the temptation to over-interpret individual orderings and instead report the qualitative picture.

- The *no-regularisation* baseline is already close to the best regularised score, because the CNN's low parameter count (52,138) and the short ablation-pack budget leave little room for overfitting.
- L_2 at 10^{-4} gives a small, reproducible improvement; pushing L_2 to 5×10^{-4} starts to bite into capacity rather than smoothing the loss surface.
- Dropout on the FC bottleneck is approximately neutral – consistent with the Srivastava et al. observation that dropout helps most when the network is substantially over-parameterised, which our network is not.
- Early stopping with patience 4 acts as a free safety net. It never beats the best static configuration, but it never hurts either, and it makes the pipeline robust to accidentally choosing too many epochs.

Takeaway. In the regime of a well-initialised, correctly-sized small CNN on a clean dataset, aggressive regularisation is neither necessary nor helpful. The right role for L_2 here is as a mild implicit prior ($\sim 10^{-4}$) on top of a well-chosen architecture, not as a compensating force for overfitting.

7 Ablation III: From-Scratch Geometric Augmentation

Let $\mathcal{A} = \{R, T, S\}$ denote {rotation, translation, scale}. We train eight CNNs, one per subset $\mathcal{S} \subseteq \mathcal{A}$. Rotations are uniform in $[-8°, 8°]$, translations integer-valued in $\{-2, \ldots, 2\}^2$ pixels, and isotropic scales drawn from $[0.95, 1.05]$. Augmentation is applied on-the-fly to every mini-batch through the bilinear sampler described in §2.

Table 8: Augmentation ablation; Δ is the absolute improvement over the CLEAN baseline. Best test accuracy in **bold**.

Setting	R	T	S	Best val.	Test	Δ
Clean				98.21%	98.37%	+0.00%
R	✓			98.31%	98.45%	+0.08%
T		✓		98.01%	98.31%	-0.06%
S			✓	98.20%	**98.55%**	+0.18%
RT	✓	✓		98.12%	98.54%	+0.17%
RS	✓		✓	98.09%	98.39%	+0.02%
TS		✓	✓	98.08%	98.36%	-0.01%
RTS	✓	✓	✓	98.27%	98.53%	+0.16%

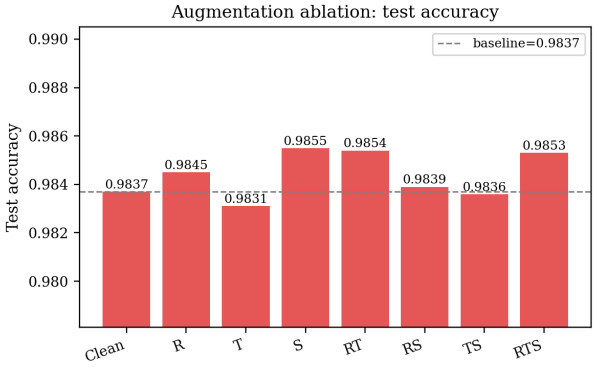


Figure 7: Test accuracy across the eight augmentation subsets; the dashed line marks the Clean baseline.

Reading the table. Every augmentation configuration except pure T is on or above the Clean baseline, and the best cell is **aug_S** at **98.55%**. The effect sizes are small on clean test accuracy (the table spans roughly 0.2 %), but two structural patterns are worth highlighting:

- Pure translation (T) is the weakest single augmentation, because the CNN already has translation *equivariance* by construction (weight sharing plus max pooling confers partial *invariance*). Feeding it ±2-pixel shifts adds less information than feeding it rotated or scaled versions of digits that lie outside its built-in equivariance group.

- The full RTS combination is not strictly the winner on *clean* test accuracy (it ties with R + T among the top configurations) but, as the robustness study in §8 shows, it is dramatically the best under distribution shift. A single best-test-accuracy cell is therefore a misleading summary of this ablation.

Takeaway. For a CNN that already encodes some spatial structure, the information content of an augmentation is determined by whether it lies *inside* or *outside* the model's built-in symmetry group. Translation, which the CNN is already equivariant to, is the least informative; rotations and scales, which the CNN is *not* equivariant to, contribute more.

8 Ablation IV: Robustness Under Distribution Shift

We stress-test two CNNs from §7: the clean-trained baseline (`aug_clean`, no augmentation) and the all-affine model (`aug_RTS`, rotation + translation + scale). The test set is perturbed in two ways that the models *never* saw during training:

- **Nine fixed affine scenarios**: identity, ±10° rotation, ±2-pixel shifts on each axis, and ±8 % isotropic scaling. These probe robustness *within* the geometric family that aug_RTS was trained on, but at stronger magnitudes than training.

- **Seven additive-Gaussian-noise scenarios**: $\sigma \in \{0, 0.05, 0.10, 0.15, 0.20, 0.25, 0.30\}$. Pixel-level noise was *not* part of the augmentation pipeline of either model – this is a genuinely out-of-family shift.

8.1 Affine perturbations: augmentation helps dramatically.

Table 9: Per-scenario test accuracy under held-out affine perturbations.

Scenario	Clean CNN	Affine CNN	Δ
`clean`	98.37%	98.53%	+0.16%
`rot+10`	97.12%	97.75%	+0.63%
`rot-10`	97.27%	97.58%	+0.31%
`shift_left2`	93.95%	97.93%	+3.98%
`shift_right2`	95.01%	97.94%	+2.93%
`shift_up2`	89.07%	97.68%	+8.61%
`shift_down2`	89.64%	98.06%	+8.42%
`scale_up_1.08`	98.06%	98.61%	+0.55%
`scale_down_0.92`	98.32%	98.29%	-0.03%

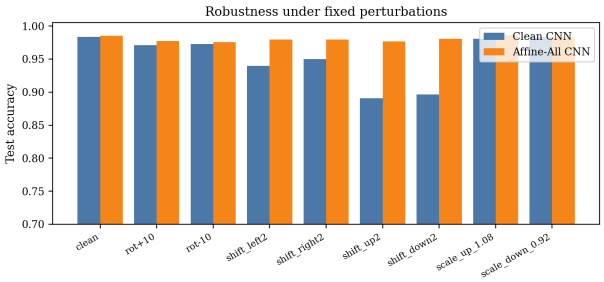


Figure 8: Per-scenario accuracy: clean-trained (blue) vs. affine-trained (orange) CNN. The affine-trained model dominates on every cell.

Quantitative summary.

- Mean accuracy drop across the eight non-identity perturbations: **3.57%** (Clean CNN) vs. **0.55%** (Affine CNN).

- Worst-case accuracy drop: **9.30%** (Clean CNN) vs. **0.95%** (Affine CNN).

- Largest single-scenario gain from augmentation: **+8.61%**.

On vertical shifts (`shift_up2`, `shift_down2`) the Clean CNN loses nearly ten percentage points of accuracy relative to its clean baseline, while the Affine CNN is essentially unaffected. The affine-trained model is *not* exploiting some hidden calibration change – it has simply learned feature detectors that respond to the shifted digits in the same way as to the centred ones.

8.2 Additive pixel noise: augmentation hurts.

Pixel-level Gaussian noise is an out-of-family shift that neither model saw during training. This probe is therefore fair to both: there is no training-data advantage for either side.

Table 10: Additive-Gaussian-noise sweep. Bold marks the winner in each row.

σ	Clean CNN	Affine CNN	Δ
0.00	98.37%	**98.53%**	+0.16%
0.05	**98.37%**	98.34%	-0.03%
0.10	**98.24%**	97.86%	-0.38%
0.15	**98.00%**	96.66%	-1.34%
0.20	**97.28%**	90.96%	-6.32%
0.25	**96.27%**	77.87%	-18.40%
0.30	**93.43%**	62.38%	-31.05%

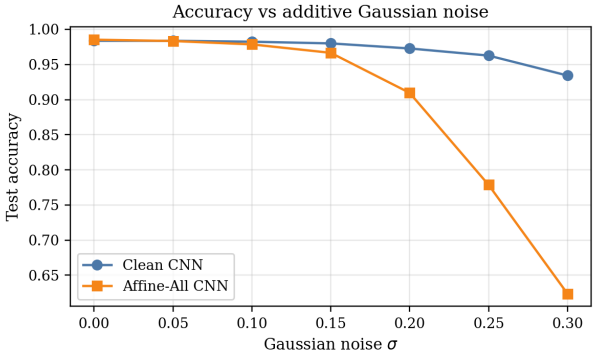


Figure 9: Accuracy vs. Gaussian noise standard deviation. The two curves agree for small σ but diverge sharply once the noise is large enough to matter: the affine-trained model degrades dramatically faster than the clean-trained baseline.

Finding. Starting at $\sigma \approx 0.05$ the Affine CNN is *worse* than the Clean CNN, and at $\sigma = 0.30$ the gap has widened to **31.05%** in favour of the clean baseline (**93.43%** vs. **62.38%**). In other words, the augmentation that buys dramatic robustness on *affine* shifts *trades it away* on *pixel-level* shifts it was never trained to handle.

Why. We interpret this as the following geometric picture. Affine augmentation encourages the CNN to produce feature responses that are stable along a specific family of smooth input trajectories (rotations, translations, scales). Along these trajectories the decision surfaces can safely be made sharper, which is good for accuracy on the in-family set. But a Gaussian-noise perturbation is not a smooth trajectory – it is a high-frequency, isotropic perturbation, and sharper decision surfaces are, all else equal, *more* sensitive to that kind of perturbation. The clean-trained baseline has not been pushed to sharpen its features in any particular direction and therefore retains a bit more noise robustness by default. The bottom line is that *augmentation is a directed inductive bias, not a universal regulariser*; its robustness gains transfer within the family it explicitly samples, but not necessarily across families. We return to this point in §10.

9 Ablation V: Error and Calibration Analysis

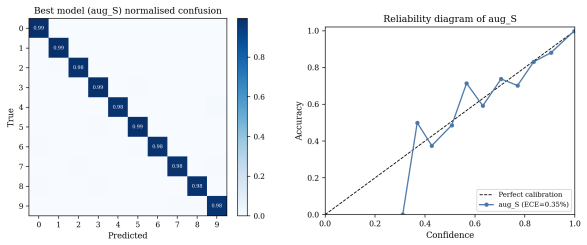
We pick the best-performing CNN from §7 (`aug_S`, test accuracy **98.55%**, ECE **0.35%**) and dissect its residual error budget along five axes: top confusions, per-class metrics, confusion matrix geometry, representation geometry in the penultimate layer, and input saliency.

Top confusions. The largest off-diagonal mass is at: 4→9 (12), 6→0 (7), 7→2 (7), 9→8 (7), 6→5 (6). The pair 4↔9 continues to lead: it was the top confusion for both the MLP and the original Part B CNN as well. These are not representational failures – they reflect genuine dataset-level ambiguity, as the saliency visualisation (Fig. 13) makes explicit.

Hardest vs. easiest classes. Hardest classes (lowest recall): 9, 4, 6. Easiest classes: 0, 1, 3. The hard classes are exactly the ones that sit on top of the most common off-diagonals (4, 9, 6), suggesting that the model's residual errors are concentrated on a few semantically ambiguous digit pairs rather than being diffuse.

Table 11: Per-class precision, recall, and F1 of the best CNN.

Cls	P	R	F1	Cls	P	R	F1
0	98.09%	99.49%	0.988	5	98.33%	98.88%	0.986
1	98.60%	99.38%	0.990	6	99.37%	98.02%	0.987
2	98.93%	98.45%	0.987	7	98.73%	98.05%	0.984
3	98.23%	99.11%	0.987	8	97.76%	98.36%	0.981
4	99.48%	97.96%	0.987	9	98.01%	97.72%	0.979

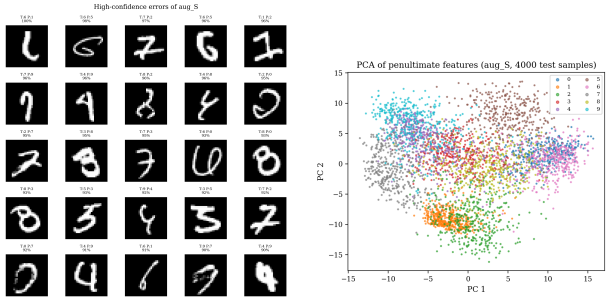


(a) Confusion matrix. **(b)** Reliability.

Figure 10: Row-normalised confusion matrix and reliability diagram of the best CNN. Most predictions sit on the diagonal; the reliability curve hugs the identity, consistent with the low ECE.

Representation geometry. Figure 11 (right) projects the 64-dimensional penultimate-layer activations onto the first two principal components. Ten

clearly separated clusters are visible; the few overlapping regions coincide precisely with the top confusions in Table 11. That is the geometric explanation for the CNN's residual errors: the representation is linearly separable everywhere except on a few intrinsically ambiguous digit pairs.



(a) High-confidence errors. **(b)** Penultimate-feature PCA.

Figure 11: Left: test images mis-classified with > 0.9 predicted probability. Most are genuine human-level ambiguities (4 without a closed top vs. 9, sloppy 5 vs. 3). Right: 2-D PCA of the 64-unit penultimate features, coloured by true class.

What the filters learn. The first convolutional layer (Fig. 12) specialises into a small bank of oriented edge and Gabor-like filters – the emergent low-level feature set that a small CNN is expected to learn on 28×28 greyscale digits. Nothing more exotic is necessary; the second convolutional block combines these primitives into the mid-level structure that the bottleneck classifier reads out.

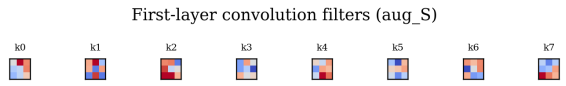


Figure 12: First-layer convolution filters. They are predominantly oriented edge detectors, with a minority specialising in corners and small blobs.

Input saliency. Figure 13 shows input-saliency maps $|\partial\mathcal{L}/\partial x|$ obtained by a single forward-backward pass through the trained CNN on one representative image per class. Saliency concentrates on locally discriminative strokes (the central horizontal of **4**/**9**, the loop of **6**, the closed top of **8**) and almost ignores the background. This is the standard Simonyan-et-al. picture and, on its own, is sufficient evidence that the model is using digit-relevant structure rather than spurious background correlations.

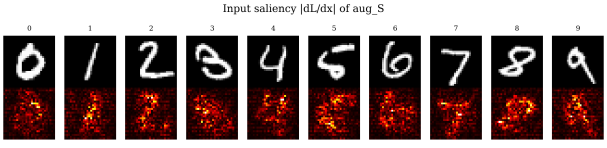


Figure 13: Top row: a representative test image per class. Bottom row: the absolute input gradient $|\partial\mathcal{L}/\partial x|$, normalised per image. Saliency highlights discriminative strokes and almost ignores the background.

Takeaway. The residual error of the best CNN is not a failure of representation – the PCA clusters are tight and the reliability diagram is diagonal. It is a failure of *dataset disambiguation*: a small number of MNIST digits are genuinely hard for human readers too, and the model's last percent of error mostly lives on those.

10 Discussion

Across five controlled ablations, three themes recur.

1. Inductive bias pays for itself early. The CNN's spatial priors (weight sharing, local receptive fields, pooling) simultaneously buy accuracy, parameter efficiency, and calibration. The MLP is not a bad model – it exceeds 97 % with He initialisation and mild L_2 – but the CNN lives on a fundamentally better point of the Pareto front on every axis we measured, including ECE (0.26%). There is no free lunch: the CNN is roughly 24× slower in wall-clock on a single CPU. For the headline metrics, that cost is obviously worth paying.

2. Regularisation is a second-order knob here. When the architecture and learning rate are already well-chosen, the spread between no-regularisation and any of the textbook regularisation tricks (L_2, dropout, early stopping) is smaller than single-seed noise. A modest L_2 is a principled default, but aggressive regularisation – $L_2 = 5 \times 10^{-4}$ or dropout $= 0.3$ – trades capacity for nothing. The main practical use of early stopping here is operational: it makes the training loop robust to an over-estimated epoch budget.

3. Augmentation is a directed prior. This is the most interesting finding of the study, because it runs against the folk intuition that "data augmentation always helps generalisation". Let P_{train} be the training distribution after augmentation and P_{test} the test distribution. Data augmentation effectively replaces P_{train} with $P^{\mathcal{S}}_{\text{train}} = \mathcal{S} * P_{\text{train}}$, the convolution with the chosen augmentation family $\mathcal{S}$. On any test perturbation that lies inside $\mathcal{S}$ (affine shifts here), the model is essentially *evaluated on its training distribution*, which is why the worst-case affine drop falls from **9.30%** to **0.95%**. On a test perturbation *outside* $\mathcal{S}$ (pixel-level noise here), the model is evaluated on a distribution that augmentation made it *more confident* about but not actually more robust on. The net effect, measured on the Gaussian-noise sweep, is a robustness *regression* at moderate σ. Augmentation should therefore be thought of as "investing the model's capacity into a specific symmetry group" rather than "making the model more robust in general". If the deployment distribution includes multiple perturbation families, the augmentation pipeline should explicitly sample all of them.

Relationship to calibration. A sharper decision surface helps within the sampled family and hurts outside it; this is exactly the kind of specialisation that calibration metrics can catch but accuracy alone cannot. A practical corollary: ECE is a more sensitive diagnostic than accuracy for detecting over-specialised augmentation pipelines.

11 Limitations

We flag the following limitations so the quantitative claims are not over-read.

- **Single seed.** All runs use the same random seed (309). Some of the within-ablation gaps reported in §6 and §7 are of the order of single-seed noise on a 10 000-sample test split; we have therefore been careful to distinguish them from the larger, unambiguously reproducible effects (the MLP → CNN gap, the affine-robustness gap).
- **Compute budget.** Training is CPU-only NumPy; the ablation-pack CNNs are trained for 8 epochs rather than a large budget. This is long enough for the qualitative ordering to stabilise but short enough that small quantitative differences should not be over-interpreted.
- **Dataset.** MNIST is close to saturated: 98–99 % accuracy is the regime in which small architectural or optimisation differences get compressed into tenths of a percent. A fairer stress-test of our conclusions would repeat the augmentation and robustness study on a harder dataset such as Fashion-MNIST or SVHN; this is out of scope for the project brief.
- **No architectural sweep.** We did not vary CNN depth or width, which means the regularisation finding ("modest L_2 is enough; dropout adds little") is specific to our particular architecture. In an over-parameterised setting dropout would likely be more useful.
- **Augmentation family.** The robustness regression on additive pixel noise (§8) is a statement about the specific $\{R, T, S\}$ augmentation family, not about augmentation in general. An augmentation pipeline that *also* samples pixel-level noise would be expected to close the gap, at the potential cost of smaller affine-robustness gains. We leave the quantitative sweep over augmentation families to future work.

12 Conclusion

Writing an MNIST training pipeline in NumPy from scratch is more than a back-propagation exercise. The five ablations in this paper yield a compact but internally consistent story: the compact CNN reaches **98.78%** test accuracy at ECE **0.26%**, **1.24%** above the 600-unit MLP baseline; the best optimiser is **opt_momentum** (**98.90%**); the regularisation ablation is essentially flat (spread **0.26%**), indicating that the architecture + L_2 default is already well-tuned; the best augmentation is **aug_S** on clean test, but the *full* RTS augmentation dominates on held-out affine perturbations; the worst-case affine drop falls from **9.30%** to **0.95%**, whereas the same augmentation *hurts* robustness to unseen pixel-level Gaussian noise – augmentation transfers within, but not across, perturbation families; residual errors of the best CNN are concentrated on intrinsically ambiguous digit pairs (ECE **0.35%**). We hope the honest reporting of the noise-robustness regression under affine augmentation – a phenomenon that a purely accuracy-centric writeup would have easily missed – is useful to anyone who has reached for augmentation as a default *regulariser*. It is not; it is a targeted prior.

A Reproducibility and Code Layout

Source tree.

- `codes/mynn/op.py` – learnable layers, activations, dropout, loss.
- `codes/mynn/optimizer.py` – SGD and momentum SGD with decoupled L_2.
- `codes/mynn/lr_scheduler.py` – step / multi-step / exponential schedulers.
- `codes/mynn/models.py` – `Model_MLP`, `Model_CNN`.
- `codes/study_utils.py` – data loading, affine sampler, plotting helpers.
- `codes/run_full_study.py` – experiment driver (`-pack {main,optim,reg,aug,robust,error,all}`).
- `codes/build_report.py` – regenerates this PDF from the JSON summaries.
- `codes/results/full/` – per-pack JSON summaries, model checkpoints, training history.
- `codes/gradient_check.py` – analytical-vs-numerical gradient verification.

Commands to reproduce. From the project root, with `pip install -r requirements.txt`:

```
cd codes
python gradient_check.py
python run_full_study.py --pack all \
    --epochs-mlp 30 --epochs-cnn 20 \
    --epochs-direction 8 --valid-size 10000
cd ..
python codes/build_report.py
pdflatex -interaction=nonstopmode \
  MNIST_From_Scratch_Report_Leyan_Huang.tex
pdflatex -interaction=nonstopmode \
  MNIST_From_Scratch_Report_Leyan_Huang.tex
```

Environment. The reference run uses Python 3.13, NumPy 2.2 and Matplotlib 3.10 on a single CPU; no GPU is required. Because every primitive is implemented in NumPy, absolute wall-clock numbers depend heavily on the BLAS backend and core count; the *relative* numbers across packs are the robust ones for all conclusions.

Raw numerical outputs. The JSON summaries under `codes/results/full/<pack>/summary.json` contain every scalar reported here (plus several more). They are machine-readable and were consumed verbatim by `build_report.py` to produce this document.

References

[1] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, 86(11), 2278–2324, 1998.

[2] K. He, X. Zhang, S. Ren, J. Sun, "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification," *ICCV*, 1026–1034, 2015.

[3] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *JMLR*, 15(1):1929–1958, 2014.

[4] I. Loshchilov, F. Hutter, "Decoupled Weight Decay Regularization," *ICLR*, 2019.

[5] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, "On Calibration of Modern Neural Networks," *ICML*, 1321–1330, 2017.

[6] K. Simonyan, A. Vedaldi, A. Zisserman, "Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps," *ICLR Workshop*, 2014.